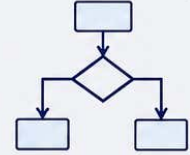
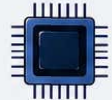


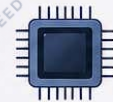
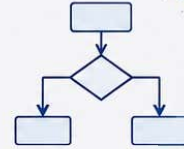
1010101010
0101010101
1010101010
0101010101



Notes



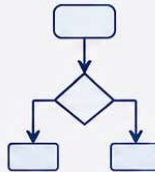
1010101010
0101010101
1010101010
0101010101



Unit-I



10110
01001
10101



Introduction & Data Preprocessing

Definition of Machine Learning

Introduction

- Machine Learning (ML) is a branch of Artificial Intelligence (AI).
- It focuses on developing algorithms that allow computers to learn from data without being explicitly programmed.
- Machine Learning helps in finding patterns in data and making predictions or decisions based on that data.
- It is widely used in applications such as image recognition, speech recognition, recommendation systems, healthcare, finance and many more.

Definition of Machine Learning

“Machine Learning is a field of study that gives computers the ability to learn from data, identify patterns and make decisions or predictions without being explicitly programmed.”

Data Preprocessing

- Data preprocessing is the process of cleaning, transforming and organizing raw data into a suitable format for machine learning models.
- It helps to improve the quality of data and the performance of the model.

Steps in Data Preprocessing:

- ① Data Cleaning – handling missing values, removing noise and duplicates.
- ② Data Integration – combining data from different sources.
- ③ Data Transformation – converting data into a suitable format (e.g., normalization, scaling).
- ④ Data Reduction – reducing the size of data while maintaining important information.

Traditional Programming vs. Machine Learning

Aspect	Traditional Programming	Machine Learning (ML)
Approach	Humans write explicit rules and logic for solving a problem.	The system learns patterns from data instead of being explicitly programmed.
Input	Data + Program (rules)	Data + Expected Output (labels)
Output	Correct output based on given rules.	A trained model that can make predictions or decisions.
Learning	Does not learn. It only follows the instructions given by the programmer.	Learns from data and improves its performance over time.
Flexibility	Not flexible. Needs to be reprogrammed for new situations.	More flexible. Can adapt to new data and make generalizations.
Example	A program to calculate area of a circle using a formula.	A model that learns to recognize cats in images from many example images.
Working Model	Input (Data) → Program (Rules) → Output (Result) Explicitly Programmed by Humans.	Input (Data) → Learning Algorithm → Output (Prediction/Decision) Learns from Data.

Types of Machine Learning : Supervised, Unsupervised, and Reinforcement Learning

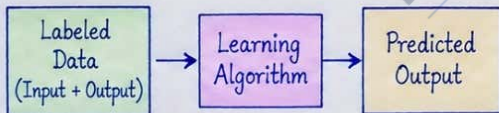
Machine Learning can be mainly classified into three types based on the type of data and learning approach.

1. Supervised Learning

- In supervised learning, the model is trained on labeled data (input with correct output).
- The goal is to learn a mapping between inputs and outputs so that it can predict the output for new, unseen data.
- It is mainly used for classification and regression problems.

Example:

Email spam detection, disease prediction, house price prediction.

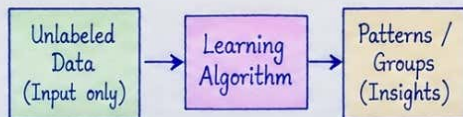


2. Unsupervised Learning

- In unsupervised learning, the model is trained on unlabeled data (only input data, without correct output).
- The goal is to find hidden patterns, structures or groupings in the data.
- It is mainly used for clustering, association and dimensionality reduction.

Example:

Customer segmentation, market basket analysis, image clustering.

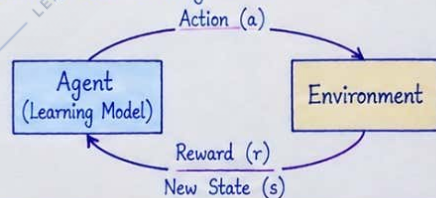


3. Reinforcement Learning

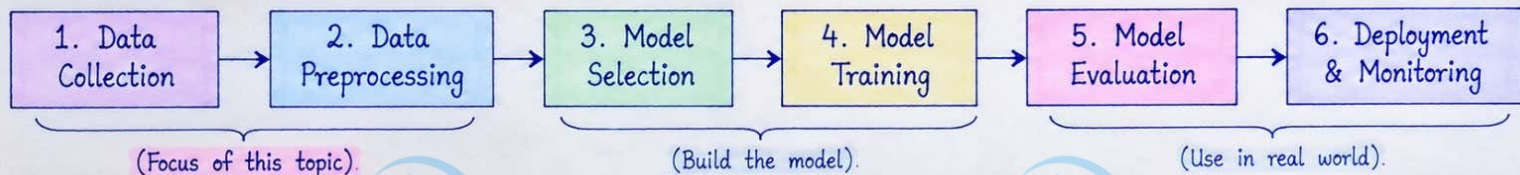
- In reinforcement learning, the model learns by interacting with an environment and receives rewards or penalties based on its actions.
- The goal is to learn a policy that maximizes cumulative reward over time.
- It is mainly used in sequential decision-making problems.

Example:

Game playing (e.g., chess, Atari games), robotics, self-driving cars.



Machine Learning Pipeline : Data Collection, Preprocessing



1. Data Collection

- Data collection is the process of gathering relevant data from different sources for a machine learning problem.
- **Sources:** databases, sensors, websites, APIs, surveys, images, videos, logs, etc.
- The quality and quantity of data directly affect the performance of the model.
- Collected data can be structured (e.g., tables) or unstructured (e.g., text, images, audio).

2. Data Preprocessing

- Data preprocessing is the process of cleaning and transforming raw data into a suitable format for machine learning models.
- Steps in data preprocessing:
 - ① Data cleaning – handling missing values, noise and duplicates.
 - ② Data integration – combining data from multiple sources.
 - ③ Data Transformation – normalization, scaling, encoding categorical data.
 - ④ Data Reduction – feature selection or dimensionality reduction (e.g., PCA).
 - ⑤ Data splitting – dividing data into training, validation and testing sets.

3. Model Selection

- Choose the appropriate algorithm (e.g., Linear Regression, Decision Tree, SVM, etc.) based on the problem and data type.

4. Model Training

- Train the selected model using the prepared training data so that it can learn patterns from the data.

5. Model Evaluation

- Evaluate the model using metrics like accuracy, precision, recall, F1-score, etc. on test data to check its performance.

6. Deployment & Monitoring

- Deploy the model in real-world applications and monitor its performance over time. Update the model with new data when needed.

A well-prepared dataset (through proper collection and preprocessing) leads to a more accurate and reliable machine learning model.

Machine Learning Pipeline : Training, Testing

1. Data Collection

2. Data Preprocessing

3. Training (Model Learning)

4. Testing (Model Evaluation)

5. Deployment & Prediction

1. Data Collection

- Gather relevant data from different sources (e.g., databases, sensors, websites, surveys).
- Data can be structured (tables) or unstructured (text, images, audio, etc.).
- Good quality and diverse data improves model performance.

2. Data Preprocessing

- Clean the data (handle missing values, remove noise, duplicates).
- Transform the data (normalization, scaling, encoding categorical variables).
- Split the dataset into training and testing sets (e.g., 80% training, 20% testing).
- Preprocessing helps in better and faster learning of the model.

3. Training (Model Learning)

- The training set is used to teach the model.
- The model learns patterns and relationships from the input data.
- An algorithm (e.g., Linear Regression, Decision Tree, Neural Network) adjusts its parameters to minimize the error (loss function).
- **Goal:** To build a model that can generalize well to unseen data.

4. Testing (Model Evaluation)

- The testing set contains unseen data that was not used during training.
- The model's performance is evaluated on this data.
- Common evaluation metrics:
 - Accuracy
 - Precision
 - Recall
 - F1-Score
 - Mean Squared Error (for regression)
- Testing helps to check how well the model performs on new, real-world data.

5. Deployment & Prediction

- Once the model performs well, it is deployed in real-world applications (e.g., web apps, mobile apps, embedded systems).
- The model takes new input data and makes predictions.
- Continuous monitoring and updates are needed to maintain good performance over time.

A well-designed pipeline with proper training and testing leads to a reliable and accurate machine learning model.

Data Preprocessing : Handling missing values, Categorical data encoding, Feature scaling, and Train-Test splitting

1. Handling Missing Values

- Missing values are empty or unknown data in a dataset.
- They can affect the performance of a machine learning model, so we need to handle them properly.

Methods:

- Remove rows/columns with missing values (if few).
- Fill with a constant value (e.g., 0 or "Unknown").
- Fill with mean, median, or mode (for numerical data).
- Use advanced methods like K-Nearest Neighbors (KNN) or interpolation.

Example:

Age	Salary	City
25	50000	Delhi
?	60000	Mumbai
30	?	Pune

 →

Age	Salary	City
25	50000	Delhi
28	60000	Mumbai
30	55000	Pune

2. Categorical Data Encoding

- Machine learning models usually work with numbers, so categorical data (text labels) must be converted into numerical format.
- Categorical data can be nominal (no order) or ordinal (with order).

Methods:

- Label Encoding – assign a unique number to each category (good for ordinal data).
- One-Hot Encoding – create binary columns for each category (good for nominal data).
- Target Encoding – replace categories with the mean of the target variable (used in some cases).

Example (One-Hot Encoding):

Color	Red	Blue	Green
Red	1	0	0
Blue	0	1	0
Green	0	0	1
Red	1	0	0

3. Feature Scaling

- Features (input variables) can have different ranges (e.g., age: 0-100, salary: 0-100000).
- Feature scaling transforms data to a similar range so that all features contribute equally to the model.
- It helps in faster training and better model performance, especially for algorithms like KNN, SVM, and Neural Networks.

Methods:

- Normalization (Min-Max Scaling) – scales values between 0 and 1.
- Standardization (Z-score Scaling) – transforms data to have mean 0 and standard deviation 1.

Example (Min-Max Scaling):

Age (Original)	Age (Scaled 0-1)
20	0.0
40	0.5
60	1.0

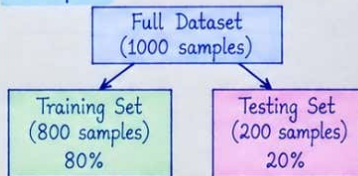
4. Train-Test Splitting

- The dataset is divided into two parts: training set and testing set.
- The model is trained on the training set and evaluated on the testing set.
- It helps to check how well the model performs on unseen data.

Common Splits:

- 80% training + 20% testing (most common)
- 70% training + 30% testing
- Other ratios (e.g., 60-40) depending on the dataset size.

Example:



Preprocessing of Image Data : Image Resizing, Grayscale Conversion

Image preprocessing is an important step in machine learning and computer vision. It helps to convert raw images into a suitable format for model training, reduces computational complexity, and improves model performance.

1. Image Resizing

- Image resizing means changing the dimensions (width $\times$ height) of an image to a fixed size.
- It is used to make all images in the dataset have the same size.
- Helps in reducing memory usage and computational cost.
- Maintains important features of the image while removing unnecessary details.

Example:



Original Image
(640 $\times$ 480)

Resize
→



Resized Image
(224 $\times$ 224)

Note: Common sizes used in deep learning models are 224 $\times$ 224, 128 $\times$ 128, or 64 $\times$ 64 pixels.

2. Grayscale Conversion

- Grayscale conversion converts a color image (with 3 channels: Red, Green, Blue) into a single channel image with different shades of gray.
- Each pixel value represents the intensity (0 = black, 255 = white).
- It reduces the amount of data and computational complexity.
- Often used when color information is not important (e.g., handwritten text, documents, edge detection).

Example:



Original Color Image
(3 channels: R, G, B)

Convert to
Grayscale
→



Grayscale Image
(1 channel)

Note: A color image has 3 channels (R, G, B), whereas a grayscale image has only 1 channel with pixel values from 0 to 255.

Converting images or video frames into arrays, Pixel Normalization

1. Converting Images or Video Frames into Arrays

- A digital image is made up of tiny squares called pixels.
- Each pixel has a numerical value representing its intensity (and color).
- So, an image can be converted into a multi-dimensional array (matrix) of numbers.
- For grayscale images $\rightarrow$ 2D array (height $\times$ width).
- For color images $\rightarrow$ 3D array (height $\times$ width $\times$ channels [R, G, B]).
- In a video, each frame is an image, so a video can be converted into a 4D array (no. of frames $\times$ height $\times$ width $\times$ channels).

Example 1: Grayscale Image



3 $\times$ 3 Image
(grayscale)

Pixel values (0-255)

34	120	200
50	80	170
10	60	90

Converted to 2D array
(shape : 3 $\times$ 3)



Example 2: Color Image (RGB)



2 $\times$ 2 Image
(color)

Converted to 3D array
(shape : 2 $\times$ 2 $\times$ 3)

$\begin{bmatrix} [255, 0, 0], & [0, 255, 0] \\ [0, 0, 255], & [255, 255, 0] \end{bmatrix}$

Video Example:

- A video with 30 frames of size 64 $\times$ 64 (grayscale) $\rightarrow$ array shape : (30, 64, 64)
- A video with 30 frames of size 64 $\times$ 64 (color) $\rightarrow$ array shape : (30, 64, 64, 3)

2. Pixel Normalization

- Pixel values are originally in the range 0 to 255 (for 8-bit images).
- Large pixel values can make model training slower and less effective.
- So, we normalize (scale) the pixel values to a smaller range, usually 0 to 1.
- This is done by dividing each pixel value by 255.
- Normalization helps the model learn faster, improves stability, and gives better performance.

Formula:

$$\text{Normalized Pixel Value} = \frac{\text{Original Pixel Value}}{255}$$

Example:

Original Image
(pixel values 0-255)

0	128
200	255

Normalized Image
(values 0-1)

0.0	0.502
0.784	1.0

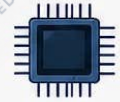
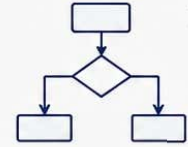


Key Points:

- Pixel normalization scales values to [0, 1].
- It prevents large numbers from affecting the learning process.
- It is a common preprocessing step for both images and video frames.

Note: The same normalization process is applied to each frame in a video before feeding it into a machine learning or deep learning model.

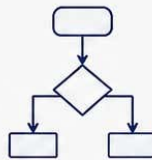
1010101010
0101010101
1010101010
0101010101



Unit - II



10110
01001
10101



Supervised Learning

Supervised Learning is a type of machine learning where the model is trained on labeled data (input along with the correct output) to learn a mapping from inputs to outputs, and then it can predict the output for new, unseen data.

Key Points

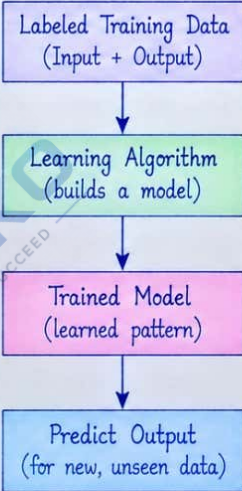
- Uses labeled data (input + correct output).
- Learns a mapping function from input to output.
- Can be used for both classification (categorical output) and regression (numerical output).
- Goal is to minimize the difference between predicted output and actual output (error).
- Common algorithms: Linear Regression, Logistic Regression, Decision Tree, Random Forest, SVM, k-NN, Neural Networks, etc.

Labeled Data Example

Input (X)	Output (Y)
House size = 1000 sq.ft	Price = 50 lakh
House size = 1500 sq.ft	Price = 75 lakh
House size = 2000 sq.ft	Price = 1 crore
House size = 2500 sq.ft	Price = 1.25 crore

Here, the input (house size) and output (price) are known, so this is labeled data.

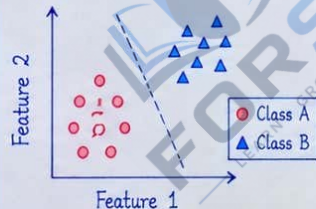
How It Works?



Types of Supervised Learning

1. Classification

- Predicts a categorical output (discrete labels).
- Examples: spam or not spam, disease or no disease, cat or dog.
- Algorithms: Logistic Regression, Decision Tree, SVM, k-NN, etc.



2. Regression

- Predicts a continuous numerical output.
- Examples: house price prediction, salary prediction, temperature prediction.
- Algorithms: Linear Regression, Decision Tree Regression, Random Forest, etc.



Applications

- Email spam detection
- Medical diagnosis
- House price prediction
- Sentiment analysis
- Image classification
- Sales forecasting, etc.

Regression : Simple and Multiple Linear Regression

Regression is a supervised learning technique used to predict a continuous numerical value. It models the relationship between a dependent variable (output) and one or more independent variables (inputs).

1. Simple Linear Regression

- Simple Linear Regression uses one independent variable (X) to predict a continuous dependent variable (Y).
- It models a linear relationship between X and Y.
- The model tries to find the best fitting straight line through the data points.
- It is simple, easy to understand and widely used for prediction.

Mathematical Model :

$$Y = \beta_0 + \beta_1 X + \epsilon$$

Y = Dependent variable (output)

X = Independent variable (input)

β_0 = Intercept (value of Y when X = 0)

β_1 = Slope (change in Y for one unit change in X)

ϵ = Error term (difference between actual and predicted value)

Example:

Predicting house price (Y) based on house size (X).



As house size increases, price also increases (linear relationship).

2. Multiple Linear Regression

- Multiple Linear Regression uses two or more independent variables to predict a continuous dependent variable.
- It models a linear relationship between the dependent variable (Y) and multiple independent variables ($X_1, X_2, \dots, X_n$).
- It can capture the effect of multiple factors on the output.
- It is widely used in real-world applications where more than one feature affects the target variable.

Mathematical Model :

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \epsilon$$

Y = Dependent variable (output)

$X_1, X_2, \dots, X_n$ = Independent variables (inputs)

β_0 = Intercept

$\beta_1, \beta_2, \dots, \beta_n$ = Coefficients (show the impact of each input variable)

ϵ = Error term

Example:

Predicting house price (Y) based on multiple features like house size (X_1), number of bedrooms (X_2), and location score (X_3).

Model :

$$\text{Price} = \beta_0 + \beta_1(\text{Size}) + \beta_2(\text{Bedrooms}) + \beta_3(\text{Location Score}) + \epsilon$$

Here, the model uses multiple factors to give a more accurate prediction than simple linear regression.

Key Takeaway:

- Simple Linear Regression → one input, predicts output.
- Multiple Linear Regression → multiple inputs, predicts output.

- Both aim to find the best fitting line (or hyperplane) that minimizes the error between actual and predicted values.

Cost Function, Basics of Gradient Descent

The cost function measures how well (or poorly) a model is performing, and gradient descent is an optimization algorithm used to minimize the cost function by iteratively updating the model parameters.

1. Cost Function

- A cost function (also called loss function) measures the difference between the predicted values and the actual values.
- It gives a single number that represents the error of the model.
- The goal is to minimize this cost function.
- For Linear Regression, the most commonly used cost function is Mean Squared Error (MSE).

Mean Squared Error (MSE):

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(ii)}) - y^{(ii)})^2$$

$J(\theta)$ = Cost function

m = Number of training examples

$h_{\theta}(x^{(ii)})$ = Predicted value

$y^{(ii)}$ = Actual value

θ = Model parameters (e.g., slope and intercept)

Key Point:

The cost function is always non-negative, and a lower value means a better model.

Example:

If the actual value is 3 and the predicted value is 5, then the squared error is:

$$(5 - 3)^2 = 4$$

This error contributes to the overall cost.

2. Basics of Gradient Descent

- Gradient Descent is an optimization algorithm used to find the values of parameters (θ) that minimize the cost function $J(\theta)$.
- It works by moving in the opposite direction of the gradient (slope) of the cost function.
- It updates the parameters iteratively until it reaches the minimum value of the cost function.
- The step size is controlled by the learning rate (α).

Parameter Update Rule:

$$\theta = \theta - \alpha \frac{\partial J(\theta)}{\partial \theta}$$

θ = Parameter (e.g., θ_0, θ_1)

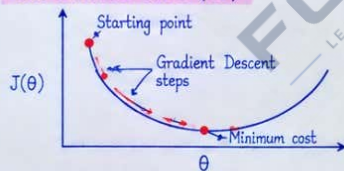
α = Learning rate (small positive value)

$\frac{\partial J(\theta)}{\partial \theta}$ = Gradient (derivative of the cost function)

Steps:

- 1 Initialize parameters (θ) with some values.
- 2 Compute the gradient (derivative of $J(\theta)$).
- 3 Update the parameters using $\theta = \theta - \alpha \frac{\partial J(\theta)}{\partial \theta}$.
- 4 Repeat steps 2 and 3 until convergence (when changes in $J(\theta)$ are very small).

Cost Function Curve (1D)



Contour Plot (2D)



Key Takeaway:

Gradient Descent helps us find the best parameters by repeatedly moving in the direction that reduces the cost function.

Classification : Logistic Regression, K-Nearest Neighbours (KNN)

Classification is a type of supervised learning where the model predicts a categorical output (discrete class label) based on input features.

1. Logistic Regression

- Logistic Regression is a classification algorithm used to predict the probability of a class (usually binary: 0 or 1).
- It uses the logistic (sigmoid) function to map any real-valued number to a value between 0 and 1.
- If the predicted probability is ≥ 0.5 , we classify it as class 1, otherwise class 0.
- It is simple, fast, and widely used in real-world applications like spam detection, disease prediction, etc.

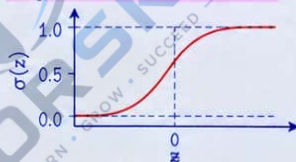
Mathematical Model :

$$P(y = 1 | x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$
$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

$\sigma(z)$ = Sigmoid (logistic) function
 z = Linear combination of input features
 β_0 = Intercept
 $\beta_1, \beta_2, \dots, \beta_n$ = Model coefficients (weights)
 $P(y = 1 | x)$ = Probability of class 1

Applications : Spam email detection, medical diagnosis, fraud detection, customer churn prediction, etc.

Sigmoid Function Curve :



Decision Rule :

If $P(y = 1 | x) \geq 0.5 \rightarrow$ Class 1
If $P(y = 1 | x) < 0.5 \rightarrow$ Class 0

2. K-Nearest Neighbours (KNN)

- K-Nearest Neighbours (KNN) is a simple, non-parametric, instance-based learning algorithm used for classification.
- It classifies a new data point based on the majority class of its k nearest neighbours in the feature space.
- It does not build an explicit model; it stores the training data and makes predictions based on similarity (e.g., Euclidean distance).
- The choice of k is important $\rightarrow$ a small k can be noisy, while a large k can over-smooth the decision boundary.

Steps of KNN :

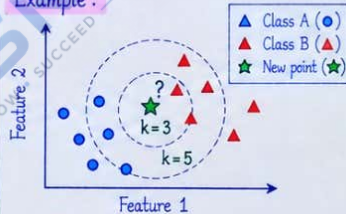
- Choose the value of k (number of neighbours).
- Calculate the distance between the new point and all training points (e.g., Euclidean distance).
- Find the k nearest neighbours.
- Assign the class based on majority voting (most common class among neighbours).

Distance Metric (Euclidean Distance):

$$d(x, x_i) = \sqrt{(x_1 - x_{i1})^2 + (x_2 - x_{i2})^2 + \dots + (x_n - x_{in})^2}$$

Applications : Image classification, recommendation systems, text classification, pattern recognition, etc.

Example :



Prediction :

For $k = 3 \rightarrow$ Class B (majority: 2 red)
For $k = 5 \rightarrow$ Class A (majority: 3 blue)

Decision Trees

Decision Tree is a supervised learning algorithm used for both classification and regression. It splits the data into subsets based on the most important feature, forming a tree-like structure of decisions.

1. Key Points

- A decision tree splits the data into smaller subsets based on feature values.
- Each internal node represents a test on a feature.
- Each branch represents the outcome of the test.
- Each leaf node gives the final prediction (class label or numerical value).
- It is simple to understand, easy to visualize, and works for both classification and regression.
- Common splitting criteria: Information Gain (Entropy), Gini Index (classification) and Mean Squared Error (regression).

2. Tree Terminology

- **Root Node** : Topmost node (entire dataset).
- **Internal Node** : A decision on a feature.
- **Branch** : Outcome of the decision.
- **Leaf Node** : Final prediction (class label or value).
- **Depth** : Longest path from root to leaf.
- **Pruning** : Removing unnecessary branches to avoid overfitting.

3. Splitting Criteria (Classification)

(a) Entropy

- Measures the impurity in a dataset.

$$\text{Entropy}(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

where, p_i = proportion of class i in dataset S .

(b) Information Gain

- Measures the reduction in entropy after a split.

$$\text{IG}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \times \text{Entropy}(S_v)$$

S = dataset, A = feature,

S_v = subset for value v ,

$|S_v|$ = number of samples in subset.

(c) Gini Index (Alternative)

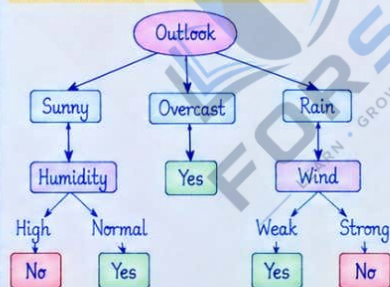
- Another measure of impurity (used in CART).

$$\text{Gini}(S) = 1 - \sum_{i=1}^c p_i^2$$

4. Example (Play Tennis Dataset)

Day	Outlook	Temp	Humidity	Wind	Play
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No

Decision Tree (Learned Model):



5. Advantages

- Easy to understand and interpret.
- Works for both classification and regression.
- Handles both numerical and categorical data.
- Requires little data preprocessing.
- Can capture non-linear relationships.

6. Disadvantages

- Prone to overfitting (can be deep and complex).
- Small changes in data can lead to a different tree (high variance).
- May not perform well on very large datasets.
- Biased toward features with more levels (in Information Gain).

7. Applications

- Spam email detection
- Medical diagnosis
- Loan approval
- Customer churn prediction
- Recommendation systems

Key Takeaway : A Decision Tree splits the data based on the most important feature (using Information Gain, Entropy, or Gini Index) to make accurate and interpretable predictions.

Naive Bayes Classifier

Naive Bayes Classifier is a simple and powerful probabilistic classification algorithm based on Bayes' Theorem. It assumes that the features are conditionally independent given the class label (hence the term "naive").

1. Key Points

- It is a supervised learning algorithm used for classification.
- Based on Bayes' Theorem.
- Assumes that all input features are independent of each other given the class label (naive assumption).
- Works well with high-dimensional data (e.g., text classification).
- Fast, simple, and effective.
- Common variants: Gaussian Naive Bayes (continuous data), Multinomial Naive Bayes (count data), Bernoulli Naive Bayes (binary data).

2. Bayes' Theorem

$$P(C | X) = \frac{P(X | C) P(C)}{P(X)}$$

Where,

$P(C | X)$ = Posterior probability (probability of class C given features X)

$P(X | C)$ = Likelihood (probability of features X given class C)

$P(C)$ = Prior probability of class C

$P(X)$ = Evidence (probability of features X)

3. Naive Assumption

For features $X = (x_1, x_2, \dots, x_n)$, we assume they are conditionally independent given the class C. So,

$$P(X | C) = \prod_{i=1}^n P(x_i | C)$$

Thus, the classifier becomes:

$$P(C | X) \propto P(C) \prod_{i=1}^n P(x_i | C)$$

We predict the class with the highest posterior probability:

$$C^* = \arg \max_C P(C) \prod_{i=1}^n P(x_i | C)$$

4. Steps of Naive Bayes Classification

- Calculate prior probabilities $P(C)$ for each class.
- Calculate likelihood probabilities $P(x_i | C)$ for each feature and class.
- Multiply the probabilities using Bayes' theorem (assuming independence).
- Predict the class with the highest posterior probability.

5. Example (Play Tennis Dataset)

Day	Outlook	Temperature	Play
1	Sunny	Hot	No
2	Sunny	Mild	No
3	Overcast	Mild	Yes
4	Rain	Cool	Yes

We want to classify a new example:

$X = (\text{Outlook} = \text{Sunny}, \text{Temperature} = \text{Cool})$

Will it Play = Yes or No ?

Step 1: Prior Probabilities

$$P(\text{Yes}) = \frac{2}{4} = 0.5 \quad P(\text{No}) = \frac{2}{4} = 0.5$$

Step 2: Likelihood Probabilities

$$P(\text{Outlook} = \text{Sunny} | \text{Yes}) = 0/2 = 0$$

$$P(\text{Outlook} = \text{Sunny} | \text{No}) = 2/2 = 1$$

$$P(\text{Temperature} = \text{Cool} | \text{Yes}) = 1/2 = 0.5$$

$$P(\text{Temperature} = \text{Cool} | \text{No}) = 0/2 = 0$$

Step 3: Compute Posterior Probabilities

$$P(\text{Yes} | X) \propto 0.5 \times 0 \times 0.5 = 0$$

$$P(\text{No} | X) \propto 0.5 \times 1 \times 0 = 0$$

Since both are 0 (due to zero counts), we may use Laplace smoothing in practice. Without smoothing, this example is hard to classify.

6. Variants of Naive Bayes

- Gaussian Naive Bayes** : Used for continuous numerical data (assumes normal distribution).
- Multinomial Naive Bayes** : Used for count data (e.g., word frequencies in text classification).
- Bernoulli Naive Bayes** : Used for binary/boolean features (e.g., word presence or absence).

7. Advantages

- Simple and easy to implement.
- Works well with small datasets.
- Effective in high-dimensional data.
- Fast training and prediction.
- Performs well in text classification (e.g., spam detection).

8. Disadvantages

- Assumes feature independence (which is rarely true in real-world data).
- Zero probability problem (solved using Laplace smoothing).
- May not perform well when features are highly dependent.

9. Applications

- Spam email filtering
- Text/document classification
- Sentiment analysis
- Medical diagnosis
- Recommendation systems

Key Takeaway: Naive Bayes is a simple yet effective probabilistic classifier that works well in many real-world applications, especially in text classification.

Support Vector Machines (SVM)

Support Vector Machine (SVM) is a supervised learning algorithm used for classification (and also regression) that finds the optimal hyperplane to separate data points of different classes with the maximum margin.

1. Key Points

- SVM is used for classification and regression (SVR).
- It finds the best (optimal) hyperplane that separates classes with the maximum margin.
- Only a few data points (support vectors) near the boundary are used to define the hyperplane.
- Works well in high-dimensional spaces.
- Effective for both linear and non-linear data (using kernel trick).
- Robust to overfitting, especially with appropriate regularization.
- Common kernels: Linear, Polynomial, RBF (Gaussian), Sigmoid.

2. Hyperplane

- A hyperplane is a decision boundary that separates different classes.
- For a 2D space, it is a line, and for a 3D space, it is a plane.
- SVM chooses the hyperplane that maximizes the margin (distance between classes).

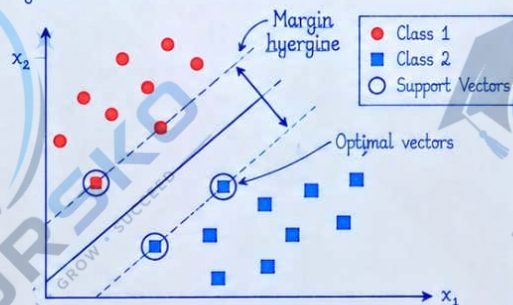
Equation of Hyperplane :

$$w \cdot x + b = 0$$

w = weight vector
 x = input feature vector
 b = bias (intercept)

3. Maximum Margin

- SVM aims to maximize the margin between the closest points (support vectors) of the two classes.
- A larger margin generally leads to better generalization.



Margin formula :

$$\text{Margin} = \frac{2}{\|w\|}$$

$\|w\|$ = magnitude of weight vector
A larger margin (smaller $\|w\|$) is preferred.

4. Kernel Trick (Non-linear Data)

- When data is not linearly separable, SVM uses a kernel function to map the data into a higher-dimensional space where it becomes linearly separable.
- Common kernels:

- Linear Kernel : $K(x, x') = x \cdot x'$
- Polynomial Kernel : $K(x, x') = (x \cdot x' + c)^d$
- RBF (Gaussian) Kernel : $K(x, x') = \exp(-\gamma \|x - x'\|^2)$
- Sigmoid Kernel : $K(x, x') = \tanh(\alpha x \cdot x' + c)$

5. SVM for Classification

- Predicts the class label based on the side of the hyperplane.
- Decision function :

$$f(x) = \text{sign}(w \cdot x + b)$$

If $f(x) > 0 \rightarrow \text{Class 1}$

If $f(x) < 0 \rightarrow \text{Class 2}$

6. Advantages and Disadvantages

Advantages :

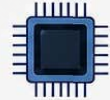
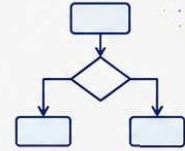
- Works well in high-dimensional data.
- Effective in cases where number of features > number of samples.
- Uses only support vectors (memory efficient).
- Good generalization performance.

Disadvantages :

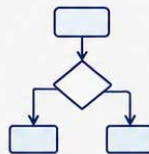
- Not suitable for very large datasets (training can be slow).
- Choice of kernel and parameters (C, γ) is important.
- Less effective when classes overlap heavily.
- Not as interpretable as decision trees.

Key Takeaway: SVM finds the optimal hyperplane with maximum margin using support vectors, and kernel functions help it handle non-linear data effectively.

1010101010
0101010101
1010101010
0101010101



10110
01001
10101



Unit - III

Unsupervised Learning : Introduction, Clustering, K-Means Clustering Algorithm

Unsupervised learning is a type of machine learning where the model learns patterns and structures from unlabeled data (i.e., data without target/output labels).

1. Introduction to Unsupervised Learning

- Unsupervised learning works with unlabeled data.
- The goal is to find hidden patterns, structure or relationships in the data.
- It learns the natural grouping or distribution of the data without any predefined output.
- Common techniques include clustering, dimensionality reduction, and association rule mining.
- Applications: customer segmentation, anomaly detection, image compression, market analysis, etc.

Supervised vs Unsupervised Learning :

Supervised Learning	Unsupervised Learning
Uses labeled data	Uses unlabeled data
Learns input-output mapping	Finds hidden patterns/structure
Examples: Classification, Regression	Examples: Clustering, Dimensionality Reduction
Output is a predicted label or value	Output is groups or structure in data

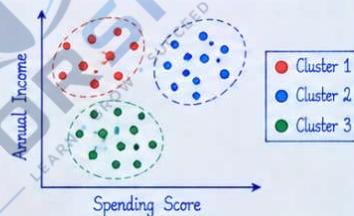
Key Point :

Unsupervised learning helps us understand the data better by finding natural groupings and patterns.

2. Clustering

- Clustering is an unsupervised learning technique that groups similar data points into clusters.
- Data points in the same cluster are more similar to each other than to those in other clusters.
- It is widely used in customer segmentation, image segmentation, document clustering, etc.
- Common clustering algorithms: K-Means, Hierarchical Clustering, DBSCAN, etc.

Example : Customer Segmentation



Key Point :

Clustering groups similar data points together based only on the input features.

3. K-Means Clustering Algorithm

- K-Means is one of the most popular and simple clustering algorithms.
- It partitions the data into K clusters, where K is a user-defined number.
- It works by minimizing the within-cluster sum of squares (WCSS), i.e., the distance between data points and their cluster centroids.
- It is iterative and converges when cluster assignments no longer change (or change very little).

Steps of K-Means Algorithm :

- Choose the number of clusters (K).
- Randomly initialize K centroids.
- Assign each data point to the nearest centroid (using Euclidean distance).
- Recalculate the centroids as the mean of data points in each cluster.
- Repeat steps 3 and 4 until the centroids do not change (convergence).

4. Mathematical Formulation

- The objective is to minimize the Within-Cluster Sum of Squares (WCSS):

$$J = \sum_{i=1}^K \sum_{x \in C_i} \|x - \mu_i\|^2$$

where,

x = data point

μ_i = centroid of cluster i

C_i = set of points in cluster i

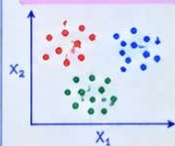
K = number of clusters

5. Example (2D Data)

Before Clustering



After K-Means (K = 3)



Advantages :

- Simple and easy to implement.
- Works well with large datasets.
- Fast and computationally efficient.

Limitations :

- K must be chosen in advance.
- Sensitive to initial centroids.
- May not work well for non-spherical clusters.
- Sensitive to outliers.

Key Takeaway : Unsupervised learning, especially clustering with K-Means, helps in discovering hidden patterns and grouping similar data points without using any labels.

Choosing Number of Clusters using Elbow Method

The Elbow Method is a simple and widely used technique to find the optimal number of clusters (K) in K-Means clustering. It works by plotting the Within-Cluster Sum of Squares (WCSS) for different values of K and looking for an "elbow" point in the plot.

1. Key Points

- The Elbow Method helps to choose the best value of K (number of clusters).
- It uses the Within-Cluster Sum of Squares (WCSS) / inertia.
- As K increases, WCSS decreases (because clusters become more compact).
- The rate of decrease is high at first, then it slows down.
- The optimal number of clusters is the point where the curve forms an "elbow" (i.e., after this point, further increase in K gives only small improvement).

2. Within-Cluster Sum of Squares (WCSS)

- WCSS measures the total squared distance between each data point and its assigned cluster centroid.
- It is also called inertia.
- Mathematically,

$$WCSS = \sum_{i=1}^K \sum_{x \in C_i} \|x - \mu_i\|^2$$

where,

K = number of clusters

C_i = set of data points in cluster i

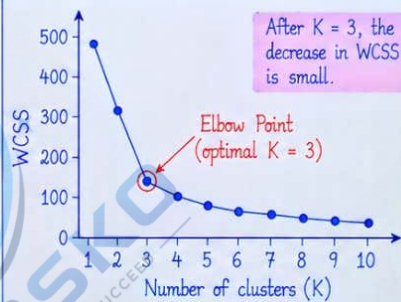
μ_i = centroid of cluster i

x = data point

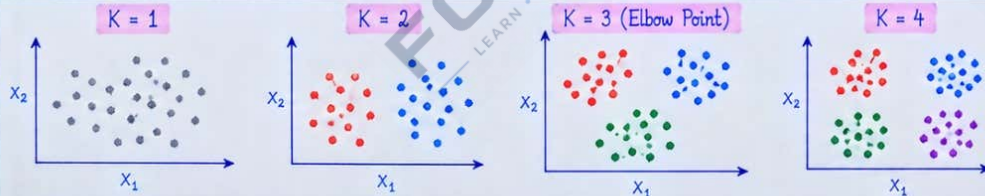
3. Steps of Elbow Method

- 1 Run K-Means for a range of K values (e.g., K = 1 to 10).
- 2 For each K, calculate the WCSS.
- 3 Plot K (x-axis) vs WCSS (y-axis).
- 4 Look for the "elbow" point in the plot.
- 5 Choose the value of K at the elbow (beyond this point, the decrease in WCSS is small).

4. Elbow Curve (Example)



5. Visual Example (2D Data)



6. Advantages and Limitations

Advantages

- Simple and easy to implement.
- Works well for many real-world datasets.
- Helps to visually find the optimal K.
- No need for labeled data.

Limitations

- Elbow point may not always be clear.
- Subjective (sometimes more than one possible elbow).
- Does not work well for complex-shaped clusters.
- Sensitive to outliers and feature scaling.

Key Takeaway: Choose the number of clusters (K) at the elbow point of the WCSS curve, where adding more clusters gives only a small improvement.

Dimensionality Reduction : Need, Introduction to Principal Component Analysis (PCA)

Dimensionality Reduction is the process of reducing the number of features (dimensions) in a dataset while preserving the most important information.

1. Need for Dimensionality Reduction

- Real-world data often has a large number of features (high-dimensional data).
- High dimensionality can lead to the "curse of dimensionality", where the data becomes sparse and models perform poorly.
- It increases computational cost and storage requirements.
- It may cause overfitting in machine learning models.
- It helps in visualization by reducing data to 2D or 3D.
- It removes redundant and irrelevant features, keeping only the most important information.
- It can improve model performance and training speed.

Examples :

- Image processing (reducing pixels)
- Text analysis (reducing word features)
- Bioinformatics (gene expression data)
- Visualization (2D/3D plots)

2. Introduction to PCA

- Principal Component Analysis (PCA) is a popular linear dimensionality reduction technique.
- It transforms the original correlated features into a new set of uncorrelated features called Principal Components (PCs).
- These principal components are linear combinations of the original features.
- PCA keeps the components that capture the most variance (information) in the data and discards the less important ones.
- It is an unsupervised learning technique.
- It is widely used in data visualization, noise reduction, and feature extraction.

Principal Component (PC):

A new feature that is a linear combination of the original features.

$$Z_1 = w_1 x_1 + w_2 x_2 + \dots + w_n x_n$$

where, $w_1, w_2, \dots, w_n$ are weights (loadings).

3. Steps of PCA

① Standardize the data (mean = 0, standard deviation = 1).

② Compute the covariance matrix.

$$C = \frac{1}{n-1} X^T X$$

where, X is the standardized data matrix.

③ Find the eigenvalues and eigenvectors of the covariance matrix.

④ Sort the eigenvectors based on decreasing eigenvalues.

⑤ Select the top k eigenvectors (principal components).

⑥ Project the original data onto the new lower-dimensional space.

Projected data :

$$Z = X W$$

where,

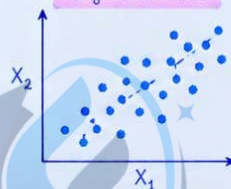
X = original standardized data ($n \times d$)

W = matrix of selected eigenvectors ($d \times k$)

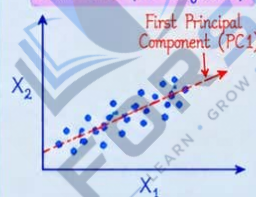
Z = transformed data ($n \times k$)

4. Visual Example (2D → 1D)

Original 2D Data



After PCA (1D Projection)



The data is projected onto the direction of maximum variance (PC1), reducing the 2D data to 1D.

5. Advantages and Limitations

Advantages :

- Reduces the number of features while retaining most of the information.
- Helps in visualization (2D or 3D).
- Removes noise and redundant features.
- Improves model performance and reduces overfitting.
- Reduces computational cost.

Limitations :

- It is a linear technique (may not capture non-linear relationships).
- The principal components are not easily interpretable in real-world terms.
- Sensitive to the scale of the data (so standardization is required).
- May not perform well if important information lies in non-linear structures.

Applications :

- Data visualization
- Image and signal processing
- Text and document analysis
- Bioinformatics
- Preprocessing for machine learning models

Key Takeaway :

PCA reduces the dimensionality of data by finding new uncorrelated features (principal components) that capture the maximum variance, helping in better visualization, faster computation, and improved model performance.

Image Feature Extraction : Basics Of image data, Converting images into arrays

Image feature extraction starts with understanding how an image is represented in a computer. An image can be converted into a numerical array (matrix) of pixel values, which can then be used to extract useful features.

1. Basics of Image Data

- An image is a grid of tiny dots called pixels (picture elements).
- Each pixel has a color (intensity) value.
- An image is represented as a 2D matrix (or 3D matrix for color images).
- Common image types :
 - Grayscale image : 1 channel (intensity)
 - Color image : 3 channels (Red, Green, Blue - RGB)
- The pixel values are usually in the range 0 to 255 (for 8-bit images).

2. Types of Images

Grayscale Image
(2D array)



- Each pixel has one value (0-255).
- Shape : (height, width)

Color Image (RGB)
(3D array)



- Each pixel has three values (R, G, B).
- Shape : (height, width, 3)

3. Converting Images into Arrays

- A digital image can be read using libraries like OpenCV, PIL (Python Imaging Library) or NumPy.
- The image is converted into a NumPy array, where each element represents a pixel value.
- For a grayscale image, the array is 2D.
For a color image, the array is 3D (with 3 channels).

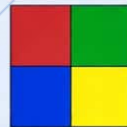
Example : Grayscale Image (4 × 4)



Array Representation :

52	55	61	66
70	68	64	60
58	55	50	45
40	42	46	48

Example : Color Image (2 × 2)



Array Representation (Shape : 2 × 2 × 3) :

$\begin{bmatrix} 255 & 0 & 0 \\ 0 & 0 & 255 \end{bmatrix}$	$\begin{bmatrix} 0 & 255 & 0 \\ 255 & 255 & 0 \end{bmatrix}$
--	--

4. Steps to Convert an Image into an Array (Python)

```
import cv2
import numpy as np

# Read image
img = cv2.imread("image.jpg") # Color image
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) # Grayscale

# Convert to array (already in numpy array format)
print(img.shape) # (height, width, 3)
print(gray.shape) # (height, width)
```

The image is automatically converted into a NumPy array when read using OpenCV.

5. Pixel Value Interpretation

- In a grayscale image, each number represents the brightness (0 = black, 255 = white).
- In a color image, each pixel is represented by three numbers (Red, Green, Blue), each ranging from 0 to 255.
- The array form allows the image to be processed, analyzed, and used for feature extraction (e.g., edges, textures, shapes).

6. Applications

- Image classification (e.g., cat vs dog).
- Object detection and recognition.
- Face recognition.
- Medical image analysis (e.g., X-rays, MRI).
- Feature extraction (edges, textures, color histograms).

Key Takeaway : An image can be converted into a numerical array of pixel values, which is the first step toward extracting useful features for machine learning and computer vision tasks.

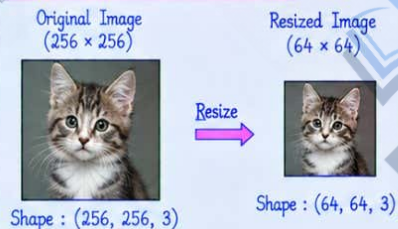
Resizing Images, Extracting Basic Features from Image Data for Classification Models

Before using images in a classification model, we usually resize them to a fixed size and extract important features (like color, edges, texture, etc.) that help the model understand and distinguish between different classes.

1. Resizing Images

- Images in real-world datasets can have different sizes.
- We resize all images to a fixed size so that they can be used as input to a machine learning model.
- Common sizes : 64×64 , 128×128 , 224×224 (depending on the model).
- Resizing reduces computational cost and keeps the input size consistent.

Example : Resizing an Image

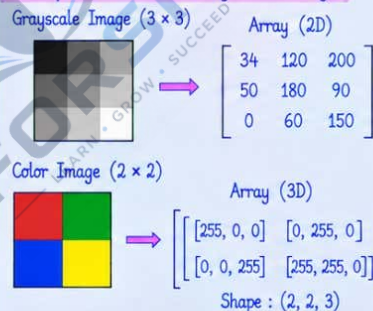


Note : Resizing can be done using libraries like OpenCV, PIL or TensorFlow.

2. Converting to Array

- A digital image is a matrix (array) of pixel values.
- For a grayscale image, it is a 2D array (height $\times$ width).
- For a color image (RGB), it is a 3D array (height $\times$ width $\times$ 3) where 3 represents Red, Green, and Blue channels.
- Each pixel value ranges from 0 to 255 (for 8-bit images).

Example : Convert Image to Array



3. Extracting Basic Features

- We can extract simple but useful features from image data for classification.
- Common basic features :

a) Color Features

- Mean color (average R, G, B values)
- Color histogram (distribution of colors)
- Helps in distinguishing objects based on color.

b) Edge Features

- Edges show the boundaries of objects.
- Can be extracted using edge detection methods (e.g., Sobel, Canny).
- Helps identify shapes and structures.

c) Texture Features

- Texture describes the surface pattern (smooth, rough, etc.).
- Common method : Local Binary Pattern (LBP).
- Helps in distinguishing objects with different textures.

d) Shape Features

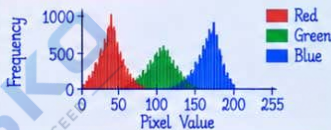
- Shape information like area, perimeter, aspect ratio.
- Can be extracted from contours.
- Useful for object recognition.

4. Example Features from an Image

Input Image
(64 x 64)



1. Color Histogram



2. Edge Image (Canny Edge Detection)



3. Texture (LBP)



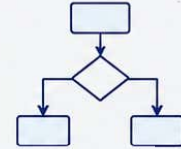
4. Shape Features (Example Values)

- Area : 1045
- Perimeter : 210
- Aspect Ratio : 1.5
- Number of Contours : 1

Key Takeaway :

Images are resized to a fixed size, converted into numerical arrays, and basic features like color, edges, texture, and shape are extracted. These features can be used as input for classification models to identify and distinguish between different classes.

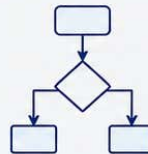
1010101010
0101010101
1010101010
0101010101



Unit - IV



10110
01001
10101



Neural Networks & Model Evaluation

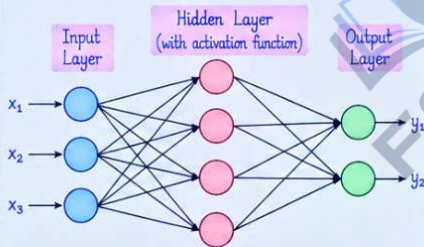
1. Neural Networks

1.1 Introduction

- Neural Networks are machine learning models inspired by the structure and working of the human brain.
- They consist of interconnected nodes called neurons organized in layers.
- Neural Networks can learn complex patterns from data and are widely used in classification, regression, image recognition, natural language processing, etc.

1.2 Structure of a Neural Network

- Input Layer : receives input features.
- Hidden Layer(s) : perform computations using weights and activation functions.
- Output Layer : produces the final prediction.



1.3 Neuron Computation

- Each neuron computes a weighted sum of inputs, adds a bias, and then applies an activation function.

$$z = \sum_{i=1}^n w_i x_i + b$$
$$a = f(z)$$

where,
 x_i = input features
 w_i = weights
 b = bias
 $f(z)$ = activation function
 a = output of neuron

1.4 Common Activation Functions

- Sigmoid : $f(z) = \frac{1}{1 + e^{-z}}$ (outputs between 0 and 1)
- ReLU : $f(z) = \max(0, z)$ (widely used)
- Tanh : $f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ (outputs between -1 and 1)

1.5 Training the Network

- The model learns by adjusting weights and bias to minimize the loss function.
- Common algorithm : Backpropagation with Gradient Descent.
- Loss Function (for classification) :

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

where, y_i = true label $\hat{y}_i$ = predicted probability

2. Model Evaluation

2.1 Introduction

- Model evaluation is the process of assessing how well a machine learning model performs on unseen data.
- It helps us to check the accuracy, reliability, and generalization ability of the model.
- A good evaluation ensures that the model is not overfitting or underfitting.

2.2 Evaluation Metrics (for Classification)

- Accuracy : proportion of correct predictions.
$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$
- Precision : proportion of true positives among predicted positives.
$$\text{Precision} = \frac{TP}{TP + FP}$$
- Recall (Sensitivity) : proportion of true positives among actual positives.
$$\text{Recall} = \frac{TP}{TP + FN}$$
- F1-Score : harmonic mean of precision and recall.
$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

2.4 Other Evaluation Techniques

- Train-Test Split : divide data into training and testing sets.
- Cross-Validation : e.g., k-Fold Cross-Validation for more reliable results.
- ROC Curve : plots True Positive Rate vs False Positive Rate.
- AUC (Area Under Curve) : measures overall performance (0 to 1).

2.3 Confusion Matrix

		Predicted	
		Positive	Negative
Actual	Positive	TP (True Positive)	FN (False Negative)
	Negative	FP (False Positive)	TN (True Negative)

TP : correctly predicted positive
TN : correctly predicted negative
FP : incorrectly predicted positive
FN : incorrectly predicted negative

2.5 Overfitting vs Underfitting

- Overfitting : model performs well on training data but poorly on unseen data.
- Underfitting : model is too simple and does not learn the patterns properly.
- Good model : balances bias and variance and generalizes well.

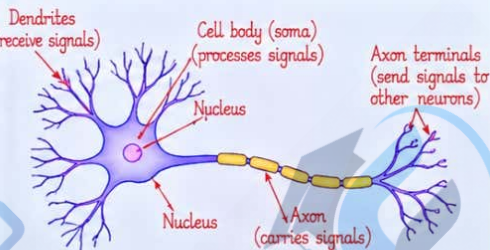
Key Takeaway :

Neural Networks learn complex patterns through interconnected neurons and backpropagation, while model evaluation helps us measure performance, avoid overfitting/underfitting, and ensure the model works well on new (unseen) data.

Neural Networks : Biological vs Artificial Neurons

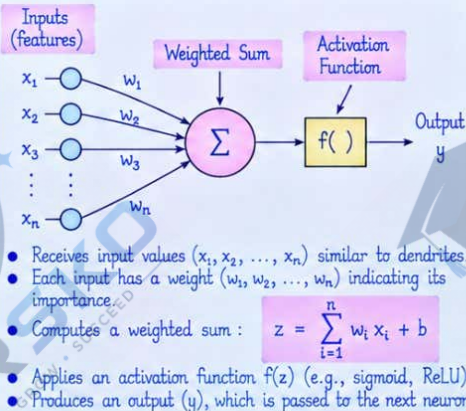
Neural Networks are inspired by the human brain. They are made up of artificial neurons that mimic the working of biological neurons, but in a simplified mathematical form.

1. Biological Neuron (Human Brain)



- Receives signals from other neurons through dendrites.
- Processes the signals in the cell body (soma).
- If the signal is strong enough, it generates an electrical impulse (action potential).
- Sends the signal through the axon to other neurons.
- Communication happens via synapses using chemical signals (neurotransmitters).

2. Artificial Neuron (in Neural Network)



- Receives input values $(x_1, x_2, \dots, x_n)$ similar to dendrites.
- Each input has a weight $(w_1, w_2, \dots, w_n)$ indicating its importance.
- Computes a weighted sum :
$$z = \sum_{i=1}^n w_i x_i + b$$
- Applies an activation function $f(z)$ (e.g., sigmoid, ReLU).
- Produces an output (y) , which is passed to the next neuron.

3. Biological vs Artificial Neuron (Comparison)

Feature	Biological Neuron	Artificial Neuron
Origin	Natural (human brain)	Man-made (computer model)
Inputs	Dendrites (electrical signals)	Input values $(x_1, x_2, \dots, x_n)$
Processing	Chemical & electrical processes in cell body	Weighted sum of inputs $(z = \sum w_i x_i + b)$
Activation	Fires action potential if threshold is reached	Uses activation function (e.g., sigmoid, ReLU)
Output	Sends signal through axon	Produces output value (y)
Connection	Synapses (with neurotransmitters)	Weights $(w_1, w_2, \dots, w_n)$
Speed	Slower (biological process)	Very fast (computational)
Complexity	Highly complex and adaptive	Simplified mathematical model
Learning	Through experience and neural plasticity	Through training (adjusting weights using algorithms)

4. Similarities

- Both receive multiple inputs.
- Both process the inputs and produce an output.
- Both have a threshold/activation mechanism (to decide whether to pass the signal).
- Both are connected to many other neurons.
- Both can learn and adapt based on experience/data.

5. Differences

- Biological neurons use chemical and electrical signals, while artificial neurons use mathematical computations.
- Biological neurons are slow but highly complex, artificial neurons are fast but simplified.
- Learning in biological neurons happens naturally, while artificial neurons learn through training algorithms.
- Biological neurons are part of a living brain, artificial neurons exist in computer programs.

6. Applications of Artificial Neurons

- Image and speech recognition.
- Natural language processing.
- Medical diagnosis.
- Self-driving cars.
- Recommendation systems (e.g., Netflix, YouTube).
- Many other AI and machine learning tasks.

Key Takeaway :

An artificial neuron is a simplified mathematical model of a biological neuron. While biological neurons are complex and natural, artificial neurons capture their basic working principle and are used to build powerful machine learning models called Artificial Neural Networks (ANNs).

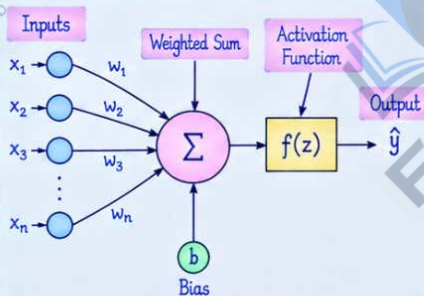
The Perceptron Model, Basic Activation Functions (Sigmoid, ReLU)

The perceptron is the simplest form of a neural network model, used for binary classification. It takes a weighted sum of inputs, applies an activation function, and produces an output. Common activation functions include Sigmoid and ReLU, which introduce non-linearity to the model.

1. What is Perceptron?

- A perceptron is a linear binary classifier proposed by Frank Rosenblatt (1958).
- It takes multiple input features, computes a weighted sum, adds a bias, and applies a step activation function.
- It classifies the input into two classes (0 or 1).
- It is the building block of neural networks.

2. Perceptron Model Diagram



3. Mathematical Model

- The perceptron computes the linear combination of inputs :

$$z = \sum_{i=1}^n w_i x_i + b$$

- Here, w_i are the weights, x_i are the inputs, and b is the bias.
- An activation function (step function) is applied to get the output :

$$\hat{y} = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{if } z < 0 \end{cases}$$

- The output $\hat{y}$ is either 0 or 1.
- The decision boundary is a linear line (or hyperplane) where $z = 0$.

4. Learning Rule (Weight Update)

- If the prediction is wrong, weights and bias are updated as:

$$\begin{aligned} w_i &= w_i + \eta (y - \hat{y}) x_i \\ b &= b + \eta (y - \hat{y}) \end{aligned}$$

- η is the learning rate.
- y is the true label (0 or 1).

5. Activation Functions

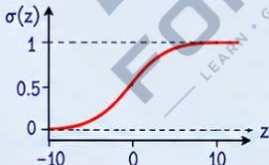
- Activation functions introduce non-linearity into the model.
- They decide the output of a neuron based on the input z .
- Common activation functions : Sigmoid, ReLU, Tanh, etc.
- In a simple perceptron, a step function is used, but in modern neural networks, we use smooth functions like Sigmoid and ReLU.

5.1 Sigmoid Activation Function

- The sigmoid function maps any real value to a range between 0 and 1.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- It is widely used in binary classification and in the output layer of neural networks.
- Smooth and differentiable.



Sigmoid Function Graph

5.2 ReLU (Rectified Linear Unit)

- ReLU is one of the most popular activation functions used in deep learning.

$$f(z) = \max(0, z)$$

- It outputs 0 for negative inputs and outputs the input itself for positive inputs.
- It is simple and computationally efficient.
- Widely used in hidden layers of neural networks.



ReLU Function Graph

Key Takeaway :

The perceptron is a simple linear classification model that uses a weighted sum and a step function. Activation functions like Sigmoid and ReLU introduce non-linearity, which helps neural networks learn complex patterns in data.

Evaluation : Confusion Matrix, Accuracy, Precision, F1-Score, Recall

Evaluation metrics are used to check how well a classification model is performing. The confusion matrix shows the counts of correct and incorrect predictions, and from it we can calculate Accuracy, Precision, Recall and F1-Score.

1. Confusion Matrix

- A confusion matrix is a table that shows the number of correct and incorrect predictions made by a classification model.
- It is mostly used for binary classification (Positive and Negative classes).
- It helps us understand the types of errors the model is making.

		Predicted Class	
		Positive	Negative
Actual Class	Positive	True Positive (TP)	False Negative (FN)
	Negative	False Positive (FP)	True Negative (TN)

Where,

- TP (True Positive) : correctly predicted positive
- TN (True Negative) : correctly predicted negative
- FP (False Positive) : incorrectly predicted positive (Type I error)
- FN (False Negative) : incorrectly predicted negative (Type II error)

2. Accuracy

- Accuracy tells us the overall proportion of correct predictions (both positive and negative).
- It works well when the classes are balanced.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

3. Precision

- Precision tells us, out of all the predicted positive cases, how many were actually positive.
- It focuses on avoiding false positives.

$$\text{Precision} = \frac{TP}{TP + FP}$$

4. Recall (Sensitivity or True Positive Rate)

- Recall tells us, out of all the actual positive cases, how many were correctly predicted.
- It focuses on avoiding false negatives.

$$\text{Recall} = \frac{TP}{TP + FN}$$

5. F1-Score

- F1-Score is the harmonic mean of Precision and Recall.
- It gives a single measure that balances both Precision and Recall.
- It is useful when the dataset is imbalanced.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

6. Example Calculation

Suppose a model gives the following confusion matrix :

		Predicted Class	
		Positive	Negative
Actual Class	Positive	50	10
	Negative	5	35

- Accuracy = $\frac{50 + 35}{50 + 35 + 5 + 10} = \frac{85}{100} = 0.85$ (85%)
- Precision = $\frac{50}{50 + 5} = \frac{50}{55} = 0.91$ (91%)
- Recall = $\frac{50}{50 + 10} = \frac{50}{60} = 0.83$ (83%)
- F1-Score = $2 \times \frac{0.91 \times 0.83}{0.91 + 0.83} = 0.87$ (87%)

7. Key Points

- Use Accuracy when the dataset is balanced.
- Use Precision when false positives are costly (e.g., spam detection).
- Use Recall when false negatives are costly (e.g., disease detection).
- Use F1-Score when you want a balance between Precision and Recall.
- Always look at the confusion matrix to understand model performance in detail.

Key Takeaway :

The confusion matrix shows the prediction results, and metrics like Accuracy, Precision, Recall and F1-Score help us evaluate the performance of a classification model from different perspectives.

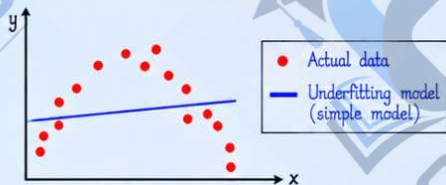
Understanding Underfitting, Overfitting, and Using Cross-validation

A good machine learning model should learn the underlying pattern in the data, generalize well to new (unseen) data, and not be too simple or too complex. Underfitting and overfitting are common problems, and cross-validation helps us evaluate the model more reliably.

1. Underfitting

- Underfitting happens when the model is too simple to learn the underlying pattern in the data.
- It has high error on both training data and test data.
- It usually occurs when the model has low complexity (e.g., linear model for nonlinear data).

Underfitting Example (High Bias)



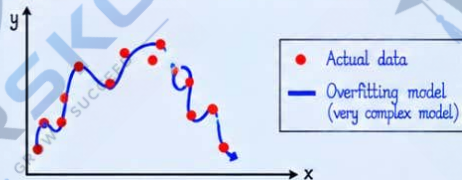
Characteristics :

- High bias
- High training error
- High test error
- Model is too simple

2. Overfitting

- Overfitting happens when the model is too complex and learn not only the underlying pattern but also the noise in the data.
- It has very low training error but high test error.
- It usually occurs when the model has high complexity (e.g., high-degree polynomial, deep neural network with too many parameters).

Overfitting Example (High Variance)



Characteristics :

- High variance
- Very low training error
- High test error
- Model is too complex

3. Using Cross-validation

- Cross-validation is a technique used to evaluate the performance of a model more reliably.
- It helps to avoid overfitting by testing the model on different subsets of the data.
- The most common method is k-Fold Cross-validation.
- The data is divided into k equal parts (folds). The model is trained on k - 1 folds and tested on the remaining fold. This process is repeated k times, and the average performance is taken.

k-Fold Cross-validation (Example: k = 5)

Iteration 1	Test	Train	Train	Train	Train
Iteration 2	Train	Test	Train	Train	Train
Iteration 3	Train	Train	Test	Train	Train
Iteration 4	Train	Train	Train	Test	Train
Iteration 5	Train	Train	Train	Train	Test

□ = Training data

□ = Testing data

Advantages :

- Gives a more accurate estimate of model performance.
- Helps in selecting the best model and hyperparameters.
- Reduces the risk of overfitting.
- Works well even with small datasets.

Key Takeaway :

Underfitting occurs when the model is too simple, overfitting occurs when the model is too complex, and cross-validation helps us evaluate the model properly and choose a model that generalizes well to new data.